

Multi-shuttle Automated Storage and Retrieval System with Quick Response Code

Pei-Chi Hu[†], Tian-Chun Chen[†], Yan-Ting Chen, Chia-Hao Cheng, Cheng-Han Li, Yun-Chien Cheng*

Department of Mechanical Engineering, National Yang Ming Chiao Tung University, Hsinchu, Taiwan

*Email: yccheng@nycu.edu.tw

[†]The authors contributed equally to this work.

Abstract

This research mainly designs a multi-shuttle Automated Storage and Retrieval System (ASRS) with the Quick Response Code (QR code) as a medium. To minimize the cost of the ASRS system, we try to complete all tasks, including website server, motor control and camera detection, by using multithreading and multiprocessing Python modules within a single board computer, a Raspberry Pi. Also, we design an algorithm to make each process work sequentially and automatically. Then we can achieve the purpose of integrating all the processes in the Raspberry Pi. The only thing that a user needs to do is to choose an inbound or outbound process for each product.

Compared with recent systems used in factories, our ASRS is more flexible. The scale of our ASRS can be large or small. If a company wants to expand its storage space, the original system does not need to be removed. We can expand the storage space based on the original structure. Therefore, because of the flexibility of the system, it can be used either in large logistics companies or small and medium enterprises. Moreover, the system can also be used in unmanned stores or e-commerce companies. Our ASRS can be widely used in many contexts.

This research aims to help warehouse owners who want to introduce an ASRS in the future. The system is low-cost, but it has many powerful functions. The owners can insert this ASRS into their original systems. Furthermore, the medium of this system, QR code, is accessible for everyone. Having this ASRS, the owners can achieve the effects of saving manpower and immediate management.

Keywords: Remote Monitoring, Multi-shuttle Automated Storage and Retrieval System, Quick Response Code

1. Introduction

Because of the COVID-19 pandemic, it is advisable to reduce one's chances of interpersonal contact. As a result, enterprises are involved in the introduction of automatic factories or automated warehouses. To make automatic factories and

warehouses possible, system integration is important, especially the integration of hardware and software.

Nowadays, the workers in factories or warehouses need to perform many tasks, such as receive, arrange, and store products. They also need to prepare orders and record product inventories every day. Fortunately, our ASRS can allow them to work more efficiently. We integrate all hardware and software with a small single-board computer, a Raspberry Pi. As a result, the workers can finish all the work by scanning QR codes in front of a computer.

In recent years, the growth of online shopping has become unstoppable, and the freight volume is increasing gradually every year. Therefore, the need for ASRS is also increasing. We aim to apply our ASRS to e-commerce stores to lower personnel costs and improve working efficiency.

2. Method

2.1 System Architecture

2.1.1 Proposed System

Our system is divided into four main processes: the inbound process, the outbound process, QR code scanning and the website. Most importantly, we hope that every process can work simultaneously. Therefore, we use the multiprocessing Python module to make these processes work independently. Each delivery has a specific QR code for identification, and each QR code contains all the information about the product inside the box. Then, we can start to give commands to our system. First, the camera scans the QR code to get the information of the product. Then the user presses the keyboard to give the inbound or outbound command. If the user presses "A", the system will start an inbound process. If the user presses "D", the system will start an outbound process. If the user presses "Q", however, the camera will stop scanning the QR code.

In the inbound process, the camera reads the information in a QR code first, and the Raspberry Pi saves the information to a MySQL database [1]. Then the website updates the data from the database in real time. After that, the box of the product is transported by an

inbound conveyor, inbound arm, inbound elevator, and inbound temporary storage conveyor, and it is finally shuttled to its specific position.

In the outbound process, the camera reads the information in a QR code first, and the Raspberry Pi finds the position of the corresponding box. Then the shuttle goes to the position of the box and retrieves it. After that, the box is transported by an outbound temporary storage conveyor, outbound elevator, and outbound conveyor. Finally, the box finishes its outbound process.

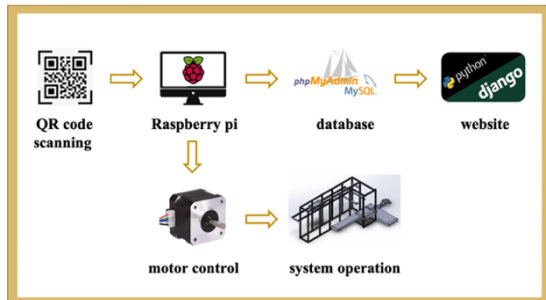


Fig. 1 The flow diagram of the system architecture.

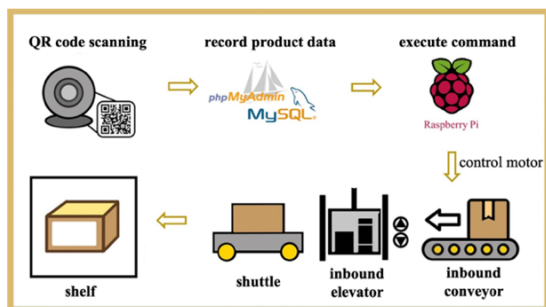


Fig. 2 The flow diagram of the inbound process.

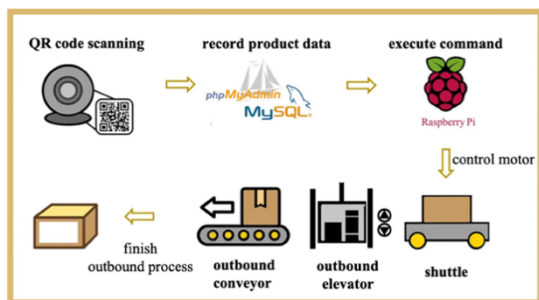


Fig. 3 The flow diagram of the outbound process.

2.1.2 Software

In the software of this system, we use several programming languages, such as PHP, MySQL, HTML, and Python, to set up a website and Graphical User Interface (GUI). The user can monitor the system on the website. The GUI provides a clear and simple interface for the user to operate.

To set up the website, we use Django, an open-source web framework written in Python. The website is connected to the database. Therefore, all data are shown on the website.

Moreover, we integrate all the software with the hardware to design a Warehouse Management System (WMS), a software package that helps manage the operations of warehouses. The website and GUI are the parts of the WMS. For example, the user can edit, search and manage the storage data online. The website shows the status of the storage in ASRS. In addition, the user can start an inbound or outbound process by scanning a QR code in the GUI. In the GUI, we use the OpenCV module to detect and read QR codes [2], and to detect keyboard inputs, we use the tkinter module [3]. Then Raspberry Pi will control all motors [4], and ASRS will start the process automatically.

To ensure that the whole system works simultaneously, we also use the multiprocessing and multithreading Python modules to allow the motors, camera, and server work independently. We separate the work into four components: inbound, outbound, scanning and website, respectively. The system follows a specially-written algorithm to complete the task sequentially and simultaneously.

2.2 Software Process

2.2.1 Process Flowchart

The process of our ASRS is mainly divided into two parts, the main process and the child process.

The main process includes QR code scanning by the camera, keyboard detection, database management, and real-time data updates on the website.

In the child process, the system fetches data from the database in Python. Then each motor works sequentially according to the command. Our system includes one main process and several child processes. The number of child processes depends on how many QR codes are scanned. That is, every component in our system can work simultaneously. Therefore, during the whole process, this system follows the specially written algorithm in the Raspberry Pi to arrange tasks.

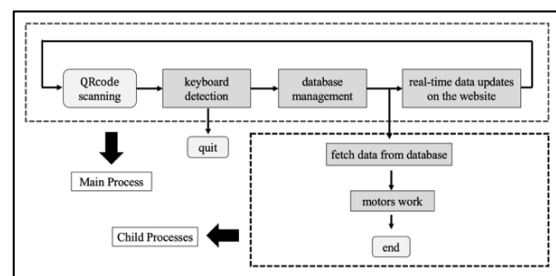


Fig. 4 The process flowchart of our ASRS.

2.2.2 Main Process

First, we use the OpenCV module to detect and read a QR code. Then we use the tkinter module to detect keyboard input. If the keyboard input signals an inbound process, the system saves the product data in the database. The product data include the inbound timestamp and other detailed information which the user adds in advance. Then the user can check the position of a product with the

GUI or detailed information by scanning a QR code. If the keyboard input signals an outbound process, the system will delete the product data in the database.

2.2.3 Child Process

There are three kinds of child processes: the shuttle process, the inbound process and the outbound process.

The shuttle process runs when the infrared sensor detects an object on the inbound temporary conveyor. The process also runs if it receives an outbound command. Then the process extracts the value from the queue of the shuttle to check whether the shuttles have tasks or not. By doing so, the shuttle executes the program once each time and operates tasks sequentially. Furthermore, we also give each shuttle a position queue to record its final position after each shuttle process. Then the program can get the position of the shuttle to run inbound or outbound processes. After that, the process returns the value to the queue of the shuttle, which means that the shuttle is free now.

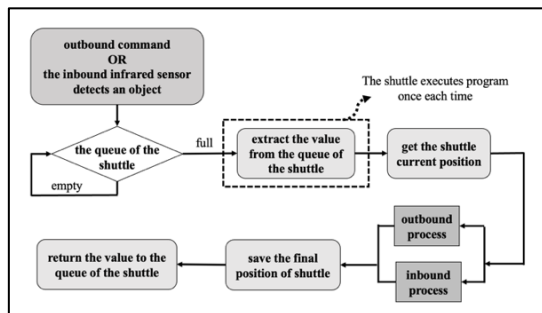


Fig. 5 The flowchart of the shuttle process.

When the system receives the inbound command, it will extract the value from the queue of the inbound process to check whether the inbound elevator is free or not. Then the inbound conveyor and inbound arm transfer the box to the inbound elevator.

In the inbound process, we use multithreading modules to control the inbound elevator and inbound temporary conveyors at the same time. Finally, the process returns the value to the queue of the inbound elevator, which means that the inbound elevator is free now.

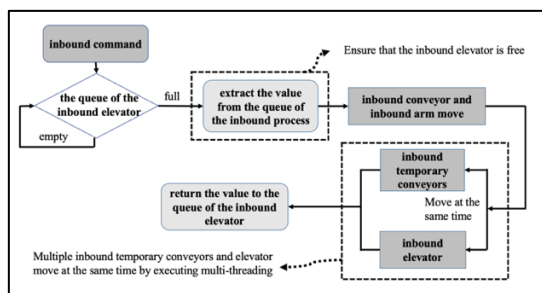


Fig. 6 The flowchart of the inbound process.

The outbound process runs when the system receives an outbound command and the infrared sensor

detects an object. Then the process extracts the value from the queue of the outbound elevator to check whether the elevator is free or not. After that, the program extracts the value from the queue of floors and completes the outbound process.

In the outbound process, we use multithreading modules to control the outbound temporary conveyors, the elevator and the outbound temporary conveyor at the same time. Finally, the process returns the value to the queue of the outbound elevator, which means the outbound elevator is free now.

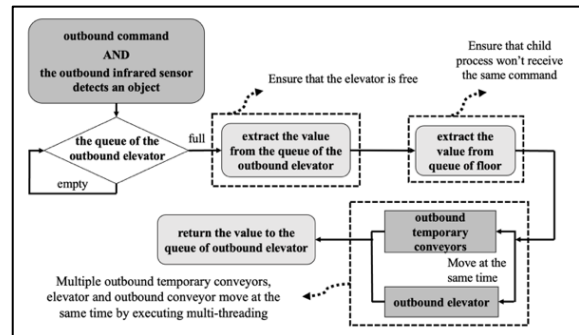


Fig. 7 The flowchart of the outbound process.

3. Results



Fig. 8 The finished ASRS.

3.1 Hardware

3.1.1 Inbound/Outbound Conveyor

The inbound and outbound conveyors are the main components of the inbound and outbound processes. They require stability and a high volume of boxes. As a result, a conveyor belt is the best choice for inbound and outbound conveyors. In terms of mechanical design, the tightness of the conveyor belt and power transmission through the belt should be considered; consequently, we design a tension device to adjust the tension of the conveyor belt. The tension device uses screws to extend a slider which is connected to the axle center of the conveyor roller, allowing adjustment of the distance between the two rollers at either end of the conveyor belt.

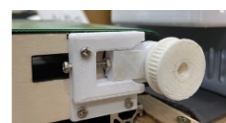


Fig. 9 The tension device on the inbound/outbound conveyor.

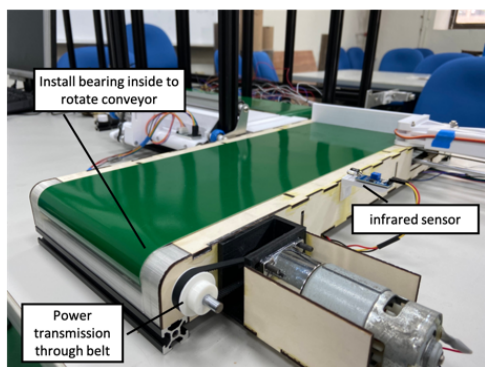


Fig. 10 The inbound conveyor.

3.1.2 Inbound Arm

The inbound arm is the device that pushes boxes from the inbound conveyor to the inbound elevator. To minimize the space that the inbound arm uses, we use three-position synthesis to design a four-bar mechanism [5].

When the infrared sensor on the inbound conveyor detects a box, the inbound arm will first check whether the box belongs to the current rack; if true, it will block the way of the box. Then, if the infrared sensor on the inbound arm detects a box, it will push the box into the elevator.



Fig. 11 The movement of the inbound arm.

3.1.3 Inbound/Outbound Elevator

The inbound elevator not only carries boxes to their own floors but also pushes boxes out of the elevator to the inbound temporary conveyor. While the elevator is moving, the center of the elevator will descend, lest boxes drop from the elevator. When the elevator is at the bottom, the center of the elevator will ascend again. By this action, the elevator and conveyor are the same height; hence, boxes can move smoothly from the conveyor to the elevator.



Fig. 12 The movement of the inbound/outbound elevators.

3.1.4 Inbound/Outbound Temporary Conveyor

To improve the efficiency of the ASRS, we add inbound and outbound temporary conveyors in this system. Shuttles can put boxes on the outbound temporary conveyor and move on to retrieve other boxes.

We use nine boxes in an outbound process as an example. The use of an outbound temporary conveyor can save about five minutes in total.

The inbound/outbound temporary conveyor is composed of two chain conveyors on the two sides. In addition, we design a slider-crank mechanism [5], a four-link mechanism that converts rotational motion to linear motion, at the end of the outbound temporary conveyor. The mechanism is used to push the boxes from the outbound temporary conveyor to the outbound elevator.

If a box is put on the temporary conveyor, the temporary conveyor will start moving until it reaches the end. Then the temporary conveyor will stop and wait for the elevator or shuttle to pick up the box.



Fig. 13 The outbound temporary conveyors with slider-crank mechanisms.

3.1.5 Shuttle

In the inbound process, shuttles lift boxes from inbound temporary conveyors and carry boxes to their positions. In the outbound process, shuttles lift boxes from the shelves and carry them to an outbound temporary conveyor.

To minimize the space that the shuttles use, we put all the extension mechanisms in the shuttle. Taking the stability of the shuttle into account, we design a two-stage extension for the extension mechanism of the shuttle. We use a linkage mechanism to realize two-stage extension with a single mechanical power output.

Compared with recent shuttles mainly used in industry, the shuttles in this research are simpler and have higher reliability. As a result, the features are easy maintenance, low cost and easy control because of the simple action of these shuttles. Moreover, the scale of these shuttles can be smaller those that of the recent shuttles. Therefore, they can be used in small storage systems.



Fig. 14 The movement of the shuttle.

3.2 Stress Analysis

Shuttle is the main component on which we perform stress analysis because its mechanism is complicated and comprises numerous small parts. Therefore, in designing shuttles, we need to consider the applied force of every part and transmission angle of the power take-off. After designing one, we do stress analysis. The results of the stress analysis show that the maximum stress of the ASRS system is less than the yield strength [6]. Therefore, the ASRS system is steady and reliable.

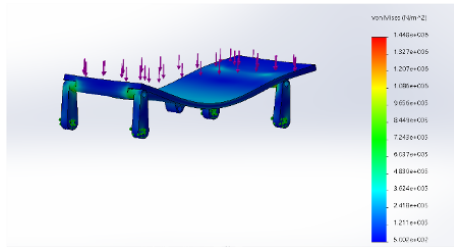


Fig. 15 The results of stress analysis of the platform of the shuttle.

3.3 Website

The user can monitor the system remotely on the website. The user can see a list of all data in the system and the detailed information about the data. Moreover, all the positions of the products are shown graphically on the website. In addition, the user can search for data information by scanning a QR code.

3.3.1 All Data Information

All Data Information shows a list of all data. It only shows three items of detailed information. Therefore, if the user wants to know more about a piece of data, he/she can click on the blue word. Moreover, the user can add more data on the website. In the list, Logdate indicates when the user adds data.

All Data Information		
Code	Name	Logdate
1001	Roseanna Quinn	2021/11/22
1002	Jaheim Stout	2021/11/22
1003	Kara Greig	2021/11/22
1004	Keniah Marriott	2021/11/22
1005	Ned Sharma	2021/11/22
1006	Kaelan Holt	2021/11/22

Fig. 16 The web page of All Data Information.

3.3.2 Detail Information

The user can click the Name button to see the Detailed Information. The user can also change the items of Detailed Information in advance. Furthermore, the user can edit or delete the data online. If the user scans a product encoded as 1001 and starts the inbound process, the timestamp will be shown in the blank of Timein,

which means the time that the product is inbound.

Detailed Information								
Code	Name	Contents	Price	Timein	Timeout	Status	Edit	Delete
1001	Peter	Red wine	500				EDIT	DELETE

Fig. 17 The web page of Detailed Information.

3.3.3 Product Position

While the box is in the inbound process, Product Position will show the position of the box in real time. As a result, the user can see the positions of all boxes by looking at Product Position and know how many vacancies exist. On the other hand, when a box is in the outbound process, Product Position will delete the position of the box. Then the system can arrange to use the position for another box.

Product Position		
3-1	3-2	3-3
2-1	2-2	2-3
1-1	1-2	1-3

Fig. 18 The web page of Product Position.

3.3.4 QR Code Search

The user can search for a piece of data by scanning a QR code. After the search, detailed information will be displayed. Then the user can go on to scan another QR code.

QR CODE SEARCH							
				SEARCH QR CODE			
				1001			
Code	Name	Contents	Price	Timein	Timeout	Logdate	Status
1001	Roseanna Quinn	Obira Granvas	500			2021/11/22	

Fig. 19 The web page of QR code Search.

3.4 System Performance

The system can manage inbound and outbound processes at the same time. The user can continue scanning QR codes in rapid succession, and the system will execute the inbound and outbound commands in sequence.

3.5 Application

The ASRS system which we have designed can be applied to a logistics center or warehouse for shops to store goods. This ASRS system allows not only automatic storage of goods but also remote monitoring of the system. This ASRS system is useful and suitable for the logistics industry because its automatic and remote data monitoring system can reduce costs, time, and manpower.

3.5.1 Application 1

Our ASRS system can be used in unmanned stores. If a customer wants to buy a product, he or she can receive the product after scanning the QR code of the product and paying the price. The store owner can check products in stock on the remote monitoring website.

3.5.2 Application 2

To apply our ASRS system to e-commerce platforms, all the owner needs to do is to install a hardware system in the storage space and connect the data to a remote monitoring website. By doing so, the sellers can send packages by QR code. After that, our ASRS system stores packages in the storage space. Then delivery personnel deliver packages to other places. Finally, the buyers can pick up their packages by scanning a QR code.

4. Conclusions

Online shopping has boomed over the last few years because of its convenience. Therefore, people have become accustomed to shopping online, and the demand for storage for this industry is sure to increase in the future.

Because of the integration of the ASRS, QR codes, website, database and WMS in one system, the user can complete all these tasks with high efficiency. The system not only saves time and manpower on operation of the system, but also allows the user to monitor the system easily and search for data conveniently. The WMS includes the website and GUI. The user can monitor the system remotely and manage the storage data. In addition, the user can give an inbound or outbound command through the GUI by scanning a QR code.

In the inbound and outbound processes, a specially-written algorithm operates the ASRS. To allow the user to enter the inbound and outbound processes in a short time, we use multiprocessing modules so that these processes are able to run sequentially. Moreover, to improve efficiency and save time, each mechanism should work independently and simultaneously. Therefore, we use multithreading and queue modules to allow each mechanism to work independently.

The challenge of this research is the integration of all the hardware and software in a single board computer, the Raspberry Pi. Furthermore, the website and database updates in real time when the system is operating.

Finally, a major feature of our ASRS is flexibility. The system can be used in storage warehouses of all

scales, from small-scale systems such as unmanned stores or e-commerce to large-scale systems such as logistics companies. Moreover, our ASRS allows a company to expand its storage space based on its original system. This feature can lighten the burden on companies intending to introduce ASRS.

5. Acknowledgements

We express our special thanks of gratitude to our thesis advisor, Dr. Yun-Chien Cheng, for his constructive suggestions and enthusiastic encouragement. Furthermore, we thank to Dr. Teng-Hu Cheng for his contributions.

We are also grateful for The Alumni Association of Mechanical Engineering, National Yang Ming Chiao Tung University, for the financial support.

Finally, we thank the anonymous reviewers and the editor for their careful reading and comments.

6. References

1. 鄧文淵 總監製 / 黃信溢、文淵閣工作室編著, 挑戰 PHP7/MySQL 程式設計與超強專題特訓班(第四版)(適用 PHP5~7, MariaDB), 2016, pp. 2-2 - 3-41、12-2 - 14-69、16-2 - 16-48
2. Andres P Jario, Source Code PH. [Online]. Available: <https://github.com/andyjario/SourceCodePH>
3. 洪錦魁, Python GUI 設計活用tkinter之路: 王者歸來, 深智數位, 2020, pp. 1-2- 4-10、17-1- 17-39
4. 王進德, Raspberry Pi 入門與機器人實作應用, 博碩文化, 2017, pp.7-92、110-159
5. Richard G. Budynas, J. Keith Nisbett "Shigley's Mechanical Engineering Design", 10th ed., Ed. New York: McGraw-Hill, 2015, pp. 43-52, 235-238, 406-411
6. Cleghorn W. L., "Mechanics of Machines", 2nd ed., Ed. New York: Oxford University Press, 2001, pp. 5-9, 53-55